

תכנות מערכות מתקדם - תרגיל מספר 1

בתרגיל זה שני חלקים

חלק א

בחלק זה עליכם ליצור פרוייקט ב JIRA, ולתכנן בו את חלק ב של תרגיל זה. תהליך הפיתוח של חלק ב יהיה בצמוד לפרוייקט ה JIRA, ואתם חייבים להשתמש ב JIRA לאורך כל הדרך בזמן אמת. הדבר יבדק על ידי הבודק.

בפרט, עליכם להקפיד על:

- חלוקה ל epics
- כתיבת users stories
- כתיבת tasks
- יצירת ספרינט, מינוי scrum master מבין חברי הקבוצה, תכנון הספרינט, קיום מפגשי סטטוס לאורך הספרינט (לא כל יום, אבל לפחות פעמיים בשבוע) חובה לתעד ב JIRA את כל הפעולות הללו ואת התוצאות שלהן. למשל, התובנות והמסקנות של מפגשי הסטטוס חייבים להיות מפורטים ומתועדים.
- עדכון סטטוס משימות (in progress, code review, done) כולל עדכון של activity של משימות תוך כדי ביצוע המשימה. שימו לב, עליכם להוסיף את הסטטוס code review, הוא לא מגיע ברירת מחדל. משימה עוברת ל code-review כאשר סיימתם אותה ופתחתם לה pull request. באפשרותכם כעת לשייך את המשימה ב JIRA אל חבר צוות אחר שצריך לעשות review לקוד שלכם. לחילופין, תוכלו להשאיר את המשימה אצלכם ול"תייג" את החבר צוות האחר ב activity של issue ב JIRA. בכל מקרה, עליכם להגדיר בצורה ברורה מי לבסוף מעביר את המשימה ל done.
- שיוך משימות לסטודנט שמבצע אותן לפני שהוא מתחיל לעבוד עליהן.
- עליכם לזהות אילו משימות "תוקעות" משימות אחרות (is blocked by) ולהביא את זה לידי ביטוי בפיציר הרלוונטי ב JIRA.
- שיוך כל המשימות ל feature branches. כלומר, יש ל Github ול JIRA התממשקות כך שאם רושמים את שם ה issue ב JIRA בשם של ה branch והמשימות, JIRA יודע להציג את זה ב JIRA issue אוטו'.

חלק ב

בחלק זה אתם מתבקשים לממש מערכת המלצה על סרטים. כלומר, כאשר משתמש מתעניין בסרט מסוים, נרצה להציע לו סרטים אחרים רלוונטיים. כיצד נמליץ? נחפש אנשים עם טעם דומה למשתמש הנוכחי שצפו בסרט הנוכחי, ולפי זה נמליץ על סרטים אחרים.

עליכם לבנות תוכנית CLI בשפת C++ אשר תשמש כמערכת המלצות סרטים.

המערכת תתמוך במספר פקודות:

```
add [userid] [movieid1] [movieid2] ...
```

הפקודה מאפשרת לשייך למשתמש (userid) רשימה של סרטים (movieids) בהם צפה. ניתן להפעיל את הפקודה כמה פעמים שרוצים על אותו userid. פקודה תקנית תכיל לפחות מזהה סרט אחד, אבל יכולה להכיל כל מספר גדול יותר. בכל פקודה, המרווח בין שדה לשדה יהיה לפחות תו רווח אחד, אך יכול להכיל יותר תווי רווח (אבל רק תווי רווח ממש, לא תוויים אחרים כמו טאב). כל מידע שמוזן בעזרת הפקודה add חייב להישמר אוטו לקובץ/קבצים. אתם יכולים לקרוא לקובץ/ים של המידע איך שתרצו. במידה והתוכנית נעצרת ומופעלת מחדש, על התוכנית לטעון את כל המידע שקיים בקובץ/ים, כך שבעצם אין משמעות לעצירת התוכנית ולהפעלתה מחדש, וההמלצות יהיו זהות בין אם הפעילו את התוכנית מחדש או לא. הסיבה היחידה שלא יהיו המלצות במערכת זה אם קבצי המידע נמחקו או רוקנו מתוכן. אין פלט לפקודה (מתקדמים לקליטת הפקודה הבאה)

recommend [userid] [movieid]

הפקודה מאפשרת להזין משתמש (userid) וסרט אחד (movieid), והמערכת תספק עד 10 המלצות לסרטים נוספים. ההמלצות יינתנו על בסיס משתמשים אחרים שהראו טעם דומה בצפייה בסרטים, לפי האלגוריתם שמוגדר בנספח לתרגיל.

help

הפקודה תדפיס למסך את רשימת כל הפקודות. כלומר, הפלט:

add [userid] [movieid1] [movieid2] ...

recommend [userid] [movieid]

help

כל פקודה תהיה בשורה אחת בלבד ותסתיים בתו ירידת שורה.
את כל הפקודות אפשר להפעיל כמה פעמים שרוצים ובאיזה סדר שרוצים.
בכל מקרה שתוזן פקודה אחרת או פקודה שאינה תקנית (למשל הפקודה foo שאינה קיימת או הפקודה add אבל בלי שום קלט נוסף באותה שורה ולכן אינה פקודה תקנית) על התוכנית להתעלם (בלי להדפיס כלום) ולהמשיך לקלוט את הפקודה הבאה. חובה להקפיד על הקלט הנ"ל במדויק ועל הפלט הנ"ל ושמופיע בנספח במדויק. אין לסטות ממנו ו/או להדפיס שום דבר נוסף/אחר/שונה, גם לא הנחיות, שגיאות או כל דבר אחר.
התוכנית לא עוצרת לעולם.

הנחיות:

- יש לתכנן את הקוד בעזרת החומר שנלמד ובדגש באופן שיאפשר הרחבה עתידית של הקוד. בפרט, יש לתת דגש על loose coupling ועל עקרונות ה SOLID. שימו לב, שבתרגילים הבאים תתבקשו לתחזק את הקוד הזה על ידי שינוי ותוספות שתתבקשו לממש. מימוש נכון, יביא לכך שיישום של הדרישות החדשות שיהיו יהיה יחסית פשוט למימוש.
- בפרט, קחו בחשבון שבתרגילים הבאים תתבקשו (לפחות):
 - לשנות את האופן שבו המידע נשמר. כלומר, במקום לשמור את המידע לקבצים ולטעון אותו מקבצים, תתבקשו לשמור את המידע במקום אחר ולטעון אותו באופן שונה.

- הקלט יגיע ממקור אחר (לא מה standard input) והפלט יהיה למקום אחר (לא אל ה standard output).
- לתמוך בפקודות אחרות.
- יש להקפיד את קוד נקי ומסודר, **מתועד**, כאשר יש הפרדה בכל המחלקות לקבצי `cpp` ו `h` והקפדה על מה צריך להיות כתוב ב `h` ומה צריך להיות כתוב ב `cpp`.
- קובץ ה README צריך להכיל הסברים מתאימים אודות האפליקציה, תמונות מתאימות, והסבר של איך מריצים ודוגמאת הרצה.
- הקפידו על חלוקה מסודרת של הפרוייקט. בתיקייה הראשית צריכה להיות תיקיית `src` אשר תכיל את הקוד שלכם ותיקיית `data` שתכיל את המידע שהקוד שלכם שומר/טוען. בנוסף, התיקייה הראשית צריכה להכיל קבצים שחייבים להיות ברמה הראשית, למשל, קובץ ה README.
- אני מזכיר, אין להעלות שום סיסמאות, מידע פרטי, רגיש או סודי, מפתחות, טוקנים, API Keys וכו' וכו' בשום מקום בפרוייקט ובקורס. לא ב `github`, לא בקוד, לא ב README, לא ב WIKI לא ב JIRA או ב DOCKER ולא בשום מקום אחר. זיכרו שלרוב הדברים יש היסטוריה - והמידע מופיע גם שם, אפילו אם מחקתם אותו.
- חובה להגיש את התרגיל בזוגות או בשלושות. הגשה ביחידים (מכל סיבה שהיא), תגורר תקרה של ציון מקסימלי של 75 בכל תרגיל שיוגש ביחידים. לא ניתן יהיה לקבל ציון מעל 75 בתרגיל שהוגש ביחידים.
- חובה לעבוד בגיט (private repository) לאורך כל התרגיל. כל סטודנט עובד מהמחשב שלו ומהמשתמש הנפרד שלו. לא ניתן לעבוד ביחד מאותו מחשב או מאותו משתמש או ללא גיט. חובה לעבוד בשיטת feature branches בלבד ולתחזק branch ראשי שאליו ממרגי'ים את ה feature branches. חובה למרגי' קוד ל branch הראשי על ידי pull request בלבד, וחובה שמאשר ה pull request יהיה שונה ממי שביקש את ה pull request (במידה כמובן שאכן הבקשה אושרה - ייתכן ומי שבוחן את בקשת ה pull ימצא בעיות במימוש שיש לתקן טרם אישור הבקשה). כל pull request חייב לעבור אישור של כל שאר חברי הקבוצה. כלומר, אם אתם זוג - אחד פותח pull request והשני עובר על הקוד ונותן הערות (אם ישנן) ורק אם הקוד תקין מאשר את ה request. אם אתם שלשה - אחד פותח pull request והשניים האחרים צריכים לעבור על הקוד ונותנים הערות (אם ישנן) ורק אם הקוד תקין שניהם מאשרים את ה request. דרך אגב, ל `github` יש פיצ'ר מובנה שמאפשר לעשות את זה. אפשר להשתמש בו אבל לא חובה. (הוא זמין בחינם רק למי שעשה את החשבון כחשבון סטודנט, ואנחנו משתמשים רק בכלים חינמיים). אם עשיתם חשבון מתאים, אתם יכולים להשתמש בזה וזה פשוט לא יאפשר למי שפתח את ה pull request לאשר לעצמו. אם בחרתם שלא להשתמש בפיצ'ר המובנה, באחריותכם להקפיד לבצע את הלוגיקה שהוגדרה (שאחד פותח PR והאחרים מאשרים) (למרגי' == לעשות merge)
- המימוש (ולכן גם תכנון המשימות ב JIRA) צריך להיות לפי TDD.
- על ההגשה להכיל Dockerfile מתאים והוראות הרצה שלו כך שניתן יהיה להריץ בעזרת Docker את התרגיל (ובנפרד, גם את ה unittests).
- את התרגיל יש להגיש למודל לתיבת ההגשה על ידי הגשת קובץ טקסט בשם `details.txt` עם שמות ות.ז. של המגישים וקישור לגיט. שימו לב, חובה על הקובץ להיות בפורמט הבא:
 - Israel Israeli 123456789
 - Israela Israeli 012345678

- Israeli Israeli 012345679
- LINK TO GITHUB HERE

בלי רווחים נוספים, בלי שורות נוספות, ובשפה האנגלית בלבד. אי הגשה של קובץ ה details.txt הנ"ל או הגשתו באופן שונה ממה שהוגדר, **תגרוור הורדה של 20 נקודות בציון התרגיל.**

- **חובה להגיש את קובץ ה details למודל מיד עם תחילת העבודה. (משימה בJIRA)**
- אנא דאגו לכך שהגיט שלכם יהיה במצב private לכל אורך הסמסטר. יש להוסיף את המייל של המתרגלים (המייל המשותף שפירסמו לכם, לא המיילים הפרטיים שלהם) עם הרשאות גישה ל Github ול JIRA.
- **אסור** להשתמש בשום סיפרייה/מודול שאינו חלק מהספרייה הסטנדרטית שמגיעה עם C++ (למעט gtest). **אין** לבקש בקשות בנוגע לתרגיל באופן מילולי. בקשות פרטניות רק דרך המייל של המתרגלים ושאלות כלליות על התרגיל אך ורק דרך הפורום. אין להשתמש בספריות שאינן טריוואליות. למשל, STL זה טריוויאלי וחלק מובנה ב C++, כך שאין בעיה להשתמש בו.
- שימו לב, אין לשנות את הrepon לאחר הדדליין. כל שינוי לאחר הדדליין ייחשב כאיחור.
- **עבודה עצמית בלבד. הנושא ייבדק ויאכף. שימו לב, התרגיל הוא שלכם ואתם צריכים לממש אותו ולא כלים מבוססים Gen AI. שימוש בכלים מסוג זה באופן שכלים אלו מממשים עבורכם חלקים מהתרגיל שקול לעבודה שאינה עצמית ועצמאית. הנושא ייבדק ויאכף.**

בהצלחה

נספח:

נתונים לדוגמה:

UserID	Movies Watched
1	100, 101, 102, 103
2	101, 102, 104, 105, 106

3	100, 104, 105, 107, 108
4	101, 105, 106, 107, 109, 110
5	100, 102, 103, 105, 108, 111
6	100, 103, 104, 110, 111, 112, 113
7	102, 105, 106, 107, 108, 109, 110
8	101, 104, 105, 106, 109, 111, 114
9	100, 103, 105, 107, 112, 113, 115
10	100, 102, 105, 106, 107, 109, 110, 116

נמצא המלצות למשתמש 1 ביחס לסרט 104.

שיטת דירוג 1:

א. נחשב את הדמיון בין משתמש 1 לבין כל משתמש אחר על פי מספר הסרטים המשותפים.

UserID	Common Movies with User 1
2	2
3	1
4	1
5	3
6	2
7	1
8	1
9	2
10	2

ב. נבדוק אילו מהמשתמשים הדומים צפו בסרט 104, ונוסיף משקל לכל סרט נוסף שהם ראו, בהתבסס על ערכי הדמיון.

1. משתמשים שצפו ב-104 הם 2, 3, 6, 8.
2. נבדוק אילו סרטים נוספים הם ראו, ונחשב משקל על ידי סכימת הדמיון:

Movielid	Users Watched (With Weights)	Total Relevance
105	2 (2), 3 (1), 8 (1)	4
106	2 (2), 8 (1)	3
107	3 (1)	1
108	3 (1)	1
109	8 (1)	1
110	6 (2)	2
111	6 (2), 8 (1)	3
112	6 (2)	2
113	6 (2)	2
114	8 (1)	1
115		0
116		0

נסביר את השורה הראשונה בטבלה.
 ב-105 צפו משתמשים 2,3,8. לכן, אנחנו סוכמים את כמות הסרטים המשותפים שלהם עם משתמש 1 (זה הערך שכתוב בסוגריים) ומקבלים ציון 4.

ג. הפלט שההמלצה צריכה לתת:

105 106 111 110 112 113 107 108 109 114

שימו לב שכאשר יש סרטים באותו ציון, מחזירים אותם לפי סדר עולה.